

FastAPI Testing — Study Summary

Summary focused on the testing concepts studied. Sections about Headers and Cookies have been removed.

1. What is Testing?

Testing means automatically sending requests to your FastAPI application and checking whether it behaves as expected. Instead of manually using a browser or Postman, you write Python test functions and let pytest run them.

Core flow:

```
Test
↓
TestClient
↓
FastAPI application
↓
Endpoint
↓
HTTP response
↓
Assertions
```

2. TestClient

TestClient is the testing HTTP client used to make requests to your FastAPI application. Create it by passing your FastAPI app to it.

Mental model:

app = the server/application being tested

client = the fake/test HTTP client

response = what the server sends back

```
from fastapi.testclient import TestClient

client = TestClient(app)

response = client.get("/items/10")
```

3. Basic Test

A test function normally starts with test_ so pytest can discover it. The tutorial uses normal def test functions and normal client calls.

```
def test_read_main():
    response = client.get("/")

    assert response.status_code == 200
    assert response.json() == {"msg": "Hello World"}
```

4. Assertions

assert checks whether a condition is true. For FastAPI tests, common checks are the HTTP status code and the JSON response body.

```
assert response.status_code == 200
assert response.json() == {"msg": "Hello World"}
```

5. Testing Errors

Good tests check successful and failing requests. The tutorial tests invalid inputs and expected HTTP errors. Check the status code and, when useful, the JSON error body.

```
response = client.get("/items/10")

assert response.status_code == 200
```

```
# Example error test
response = client.get("/items/0")
assert response.status_code == 422
```

6. Path and Query Parameters

Path parameters go directly in the URL. Query parameters go after ?. Optional query parameters can be omitted.

```
response = client.get("/items/10")
response = client.get("/items/10?q=phone")
```

7. JSON Request Bodies

Use json= to send a JSON request body.

```
response = client.post(
    "/items/",
    json={
        "id": "foobar",
        "title": "Foo Bar",
        "description": "The Foo Barbers"
    }
)

assert response.status_code == 200
```

8. Pydantic Models in Tests

TestClient expects data that can be converted to JSON, not a Pydantic model directly. If you need to send Pydantic model data, the tutorial points to jsonable_encoder for converting it into JSON-compatible data.

```
from fastapi.encoders import jsonable_encoder

json_data = jsonable_encoder(my_model)

response = client.post(
    "/items/",
    json=json_data
)
```

9. Separate Test Files

In a real application, tests are normally placed in a separate file. A common structure is:

```
app/
■■■ __init__.py
■■■ main.py
■■■ test_main.py

# test_main.py
from fastapi.testclient import TestClient
from .main import app

client = TestClient(app)
```

10. Validation Test Example

You can test FastAPI validation as well as successful requests. For example, if item_id must be at least 1, requesting /items/0 should produce a 422 validation response.

```
from fastapi import FastAPI, Path
from typing import Annotated

@app.get("/items/{item_id}")
async def get_item(
    item_id: Annotated[int, Path(ge=1)],
    q: str | None = None
):
    return {"item_id": item_id, "q": q}
```

```
def test_invalid_item_id():
    response = client.get("/items/0")
    assert response.status_code == 422
```

11. Complete Practice Example

This combines the main concepts you practiced: FastAPI, Path validation, TestClient, GET requests, query parameters, and assertions.

```
from fastapi import FastAPI, Path
from fastapi.testclient import TestClient
from typing import Annotated

app = FastAPI()

@app.get("/items/")
async def get_all_items():
    return {"message": "all items"}

@app.get("/items/{item_id}")
async def get_item(
    item_id: Annotated[int, Path(ge=1)],
    q: str | None = None
):
    return {"item_id": item_id, "q": q}

client = TestClient(app)

def test_get_item():
    response = client.get("/items/10?q=phone")
    assert response.status_code == 200
    assert response.json() == {
        "item_id": 10,
        "q": "phone"
    }

def test_get_item_without_query():
    response = client.get("/items/10")
    assert response.status_code == 200
    assert response.json() == {
        "item_id": 10,
        "q": None
    }

def test_invalid_item_id():
    response = client.get("/items/0")
    assert response.status_code == 422
```

12. Common Mistakes

- Forgetting to create TestClient(app).
- Forgetting to assert the expected status code.
- Checking only success cases and never testing expected errors.
- Using the wrong request argument for the data you want to send.
- Making a required path parameter optional unnecessarily.
- Manually validating something FastAPI can validate with Path/Pydantic.
- Putting unrelated scenarios into one huge test; separate tests make failures easier to understand.

13. Quick Cheat Sheet

Remember these patterns:

```
client = TestClient(app)

client.get("/items/10")
client.get("/items/10?q=phone")
client.post("/items/", json={...})
```

```
assert response.status_code == 200
assert response.json() == {...}

def test_something():
    ...
```

14. Final Mental Model

app = FastAPI application/server
client = test HTTP client
request = what the test sends
response = what the app sends back
assert = what the test expects

The client does not decide whether your code is correct. It sends the request and gives you the response; your test assertions decide whether the observed result matches the expected result.

```
test function
↓
TestClient sends request
↓
FastAPI validates + runs endpoint
↓
HTTP response
↓
assert status code
↓
assert response body
↓
PASS / FAIL
```

Source: uploaded FastAPI Testing tutorial.